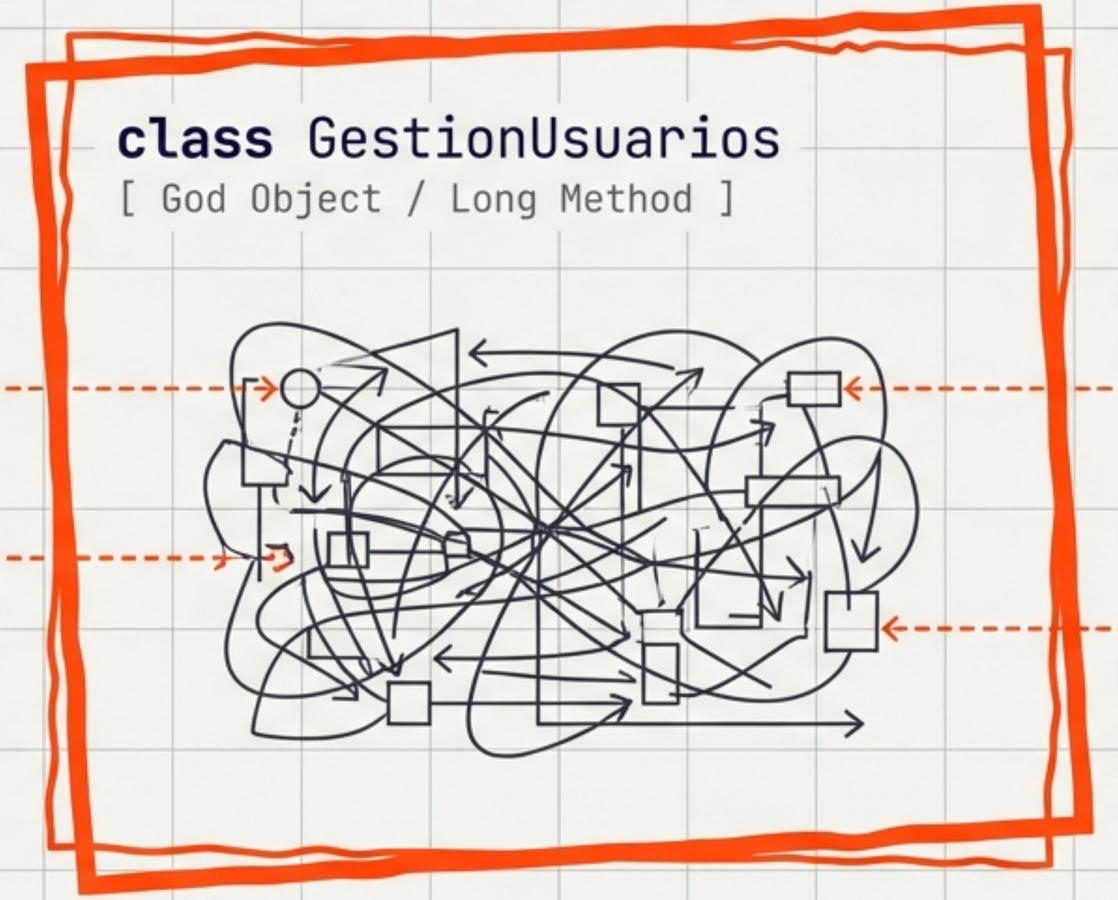
El colapso del monolito procedural

Falta de Cohesión

Métodos ajenos como `exportarLogs()` conviven forzosamente con el registro de usuarios.

Fragilidad de Datos

Un simple `List<String>` almacena únicamente nombres, perdiendo silenciosamente emails, contraseñas y edades.



Validaciones Duplicadas

La verificación del formato de email (@ y .) se repite de forma idéntica en múltiples métodos.

Control de Errores Deficiente

Saltos abruptos con `return` en medio del código, rompiendo el flujo predecible de ejecución.

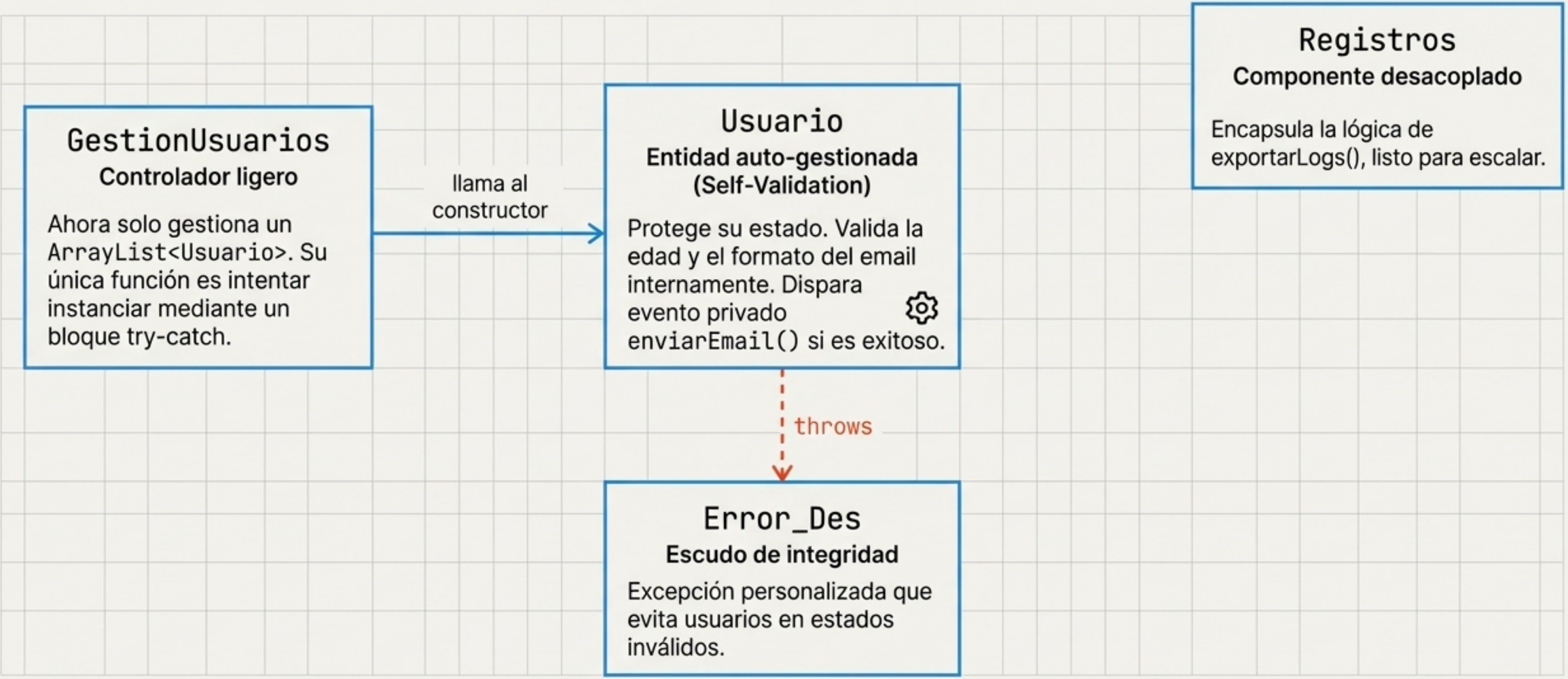
NotebookLM

Ingeniería de refactorización aplicada

Problema Detectado	Patrón Aplicado	El Resultado
Datos Primitivos (List<String>) y pérdida de contexto.	`Replace Primitive with Object`	Creación de la entidad Usuario, centralizando nombre, edad, email y contraseña.
Validaciones esparcidas y salidas abruptas.	`Single Entry, Single Exit & Self-Validation`	Flujo continuo. Las validaciones se encapsulan directamente en los Setters del objeto.
Errores genéricos incapaces de explicar fallos críticos.	`Custom / User-Defined Exception`	Nace la clase Error_Des, interceptando y explicando creaciones inválidas.
Lógica mezclada (God Object).	`Extract Class & Move Method`	Desacoplamiento total. Registros gestiona logs; Usuario maneja el Internal Event Trigger para emails.

NotebookLM

Responsabilidad Única en acción



De un script procedural frágil a un sistema escalable, cohesivo y tolerante a fallos.

NotebookLM